

A Project Report  
on  
**Credit Card Fraud Detection: Random Forest Based  
Fraud Monitoring Dashboard**

Submitted in partial fulfillment of the requirement of Computer Project-II  
(BCE5017)

of

BACHELOR IN COMPUTER ENGINEERING



Purbanchal University

Biratnagar, Nepal

**Submitted by:**

Santosh Kumar Shah (751604)

Surya Chand Yadav (752074)

Salim Shrestha (751600)

**KANTIPUR CITY COLLEGE**

Putalisadak, Kathmandu

**July, 2026**

A Project Report  
on  
**Credit Card Fraud Detection: Random Forest Based  
Fraud Monitoring Dashboard**

Submitted in partial fulfillment of the requirement of Computer Project-II  
(BCE5017)

of  
BACHELOR IN COMPUTER ENGINEERING

**Submitted to:**

Purbanchal University

Biratnagar, Nepal

**Submitted by:**

Santosh Kumar Shah (751604)

Surya Chand Yadav (752074)

Salim Shrestha (751600)

**Project Supervisor:**

Er. Sagar Dhakal

**KANTIPUR CITY COLLEGE**

Putalisadak, Kathmandu

**July, 2026**

## Declaration

We hereby declare that the project report entitled “*Credit Card Fraud Detection*”, submitted in partial fulfillment of the requirements for the degree of Bachelor of Engineering in Computer Engineering, is the result of our original work carried out under the supervision of **Er. Sagar Dhakal**.

We further declare that:

1. This work has not been submitted previously for the award of any degree, diploma, or similar title at any other institution or university. Proper acknowledgements have been given wherever the work of others has been referenced.
2. The project complies with the rules and regulations of Purbanchal University and Kantipur City College.

We understand that any violation of academic integrity may result in disciplinary action.

Santosh Kumar Shah (751604)

Surya Chand Yadav (752074)

Salim Shrestha (751600)

July, 2026

## Acknowledgement

We would like to express our sincere gratitude to all those who supported and guided us in the successful completion of this project titled “*Credit Card Fraud Detection*”.

First and foremost, we extend our appreciation to our supervisor **Er. Sagar Dhakal** for his guidance, feedback, and support throughout the development and documentation of this project.

We also thank the faculty members and staff of the Department of Computer Engineering at Kantipur City College for providing the academic environment, resources, and suggestions needed to plan, implement, test, and document this work. We are grateful to our classmates, friends, and families for their cooperation, patience, and encouragement throughout the project period.

Santosh Kumar Shah (751604)

Surya Chand Yadav (752074)

Salim Shrestha (751600)

July, 2026

## Abstract

This project presents Fraud Shield, a credit card fraud detection system that combines a trained Random Forest classifier with a Flask-based monitoring dashboard. Raw transactions are transformed into 15 engineered features, scaled, and classified to produce a fraud probability, which is combined with behavior-based signals into a risk score (0–100). The system supports single-transaction checks and CSV batch detection, along with alerts, verification workflow, and downloadable reports.

The model was trained on 59,978 synthetic transactions (`creditcard_raw.csv`), including 922 fraud cases (1.54%), using a chronological 70/15/15 split and a probability threshold of 0.371. On the test split, it achieves 98.56% accuracy, 52.22% fraud precision, 68.12% fraud recall, 65.22% PR-AUC, and 97.32% ROC-AUC. These results reflect performance on the synthetic benchmark and should not be interpreted as representative of real banking data.

[1–3].

**Keywords:** Credit card fraud detection, Random Forest, machine learning, Flask dashboard, risk scoring, transaction monitoring, behavior analysis, fraud alerts.

## Supervisor's Approval

This is to certify that the project entitled “*Credit Card Fraud Detection*”, undertaken and successfully demonstrated by **Santosh Kumar Shah, Surya Chand Yadav, and Salim Shrestha**, has been completed under my guidance.

This project is submitted as partial fulfillment of the requirements for the degree of Bachelor of Engineering in Computer Engineering under Purbanchal University. Throughout the duration of the project, the students have demonstrated dedication, technical understanding, and practical implementation skills in machine learning, web application development, testing, and documentation.

I hereby approve this project for certification by the concerned authority.

---

**Mr. Sagar Dhakal**  
Project Supervisor  
Lecturer  
Department of Computer Engineering  
Kantipur City College

Date: 31 July 2026

## Certificate from Department

This is to certify that, following the Supervisor's Approval and Examiners' Acceptance, the project entitled "*Credit Card Fraud Detection*", submitted by **Santosh Kumar Shah, Surya Chand Yadav**, and **Salim Shrestha**, has been officially approved as a partial fulfillment of the requirements for the degree of Bachelor of Engineering in Computer Engineering under Purbanchal University.

The department acknowledges the students' efforts and successful completion of the project.

---

**Er. Subash Rajkarnikar**

Head of Department,  
Department of Computer Engineering,  
Kantipur City College

Date: 31 July 2026

# Contents

|                                                               |            |
|---------------------------------------------------------------|------------|
| <b>Declaration</b>                                            | <b>ii</b>  |
| <b>Acknowledgement</b>                                        | <b>iii</b> |
| <b>Abstract</b>                                               | <b>iv</b>  |
| <b>Supervisor’s Approval</b>                                  | <b>v</b>   |
| <b>Certificate from Department</b>                            | <b>vi</b>  |
| <b>1 Introduction</b>                                         | <b>1</b>   |
| 1.1 Overview . . . . .                                        | 1          |
| 1.2 Problem Statement . . . . .                               | 1          |
| 1.3 Objectives . . . . .                                      | 1          |
| 1.4 Major Features . . . . .                                  | 2          |
| 1.5 Significance . . . . .                                    | 2          |
| 1.6 Scope . . . . .                                           | 3          |
| 1.7 Limitations . . . . .                                     | 3          |
| 1.8 Organization of the Document . . . . .                    | 3          |
| <b>2 Literature Review</b>                                    | <b>4</b>   |
| 2.1 Theoretical Review . . . . .                              | 4          |
| 2.2 Empirical Review . . . . .                                | 4          |
| 2.3 Machine Learning Techniques for Fraud Detection . . . . . | 5          |
| 2.4 Research Gaps . . . . .                                   | 5          |
| 2.5 Literature Summary . . . . .                              | 5          |
| <b>3 System Analysis</b>                                      | <b>6</b>   |
| 3.1 Existing System . . . . .                                 | 6          |
| 3.2 Proposed System . . . . .                                 | 6          |
| 3.3 Functional Requirements . . . . .                         | 6          |
| 3.4 Non-Functional Requirements . . . . .                     | 7          |
| 3.5 Feasibility Study . . . . .                               | 7          |
| 3.5.1 Technical Feasibility . . . . .                         | 7          |
| 3.5.2 Economic Feasibility . . . . .                          | 7          |
| 3.5.3 Operational Feasibility . . . . .                       | 7          |



|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| 3.5.4    | Schedule Feasibility . . . . .                     | 8         |
| 3.6      | Ethical and Security Considerations . . . . .      | 8         |
| <b>4</b> | <b>Methodology</b>                                 | <b>9</b>  |
| 4.1      | Software Development Life Cycle . . . . .          | 9         |
| 4.2      | Technology and Tools Used . . . . .                | 11        |
| 4.3      | Assignment of Roles and Responsibilities . . . . . | 11        |
| 4.4      | Dataset . . . . .                                  | 12        |
| 4.5      | Data Preparation . . . . .                         | 12        |
| 4.6      | Input Fields . . . . .                             | 12        |
| 4.7      | Feature Engineering . . . . .                      | 13        |
| 4.8      | Model Training . . . . .                           | 13        |
| 4.9      | Prediction Pipeline . . . . .                      | 14        |
| 4.10     | Risk Score Calculation . . . . .                   | 16        |
| <b>5</b> | <b>System Design</b>                               | <b>17</b> |
| 5.1      | Architecture . . . . .                             | 17        |
| 5.2      | Use Case Design . . . . .                          | 19        |
| 5.3      | Data Flow Design . . . . .                         | 20        |
| 5.4      | API Design . . . . .                               | 20        |
| 5.5      | Persistence Design . . . . .                       | 21        |
| 5.6      | User Interface Design . . . . .                    | 21        |
| 5.7      | User Interface Screenshots . . . . .               | 21        |
| <b>6</b> | <b>Implementation</b>                              | <b>24</b> |
| 6.1      | Development Environment . . . . .                  | 24        |
| 6.2      | Authentication . . . . .                           | 24        |
| 6.3      | Transaction Validation . . . . .                   | 24        |
| 6.4      | Feature Encoding . . . . .                         | 24        |
| 6.5      | Behavior Analysis . . . . .                        | 24        |
| 6.6      | Prediction and Risk . . . . .                      | 25        |
| 6.7      | Alerts and Verification . . . . .                  | 25        |
| 6.8      | CSV Batch Detection . . . . .                      | 25        |
| 6.9      | Reporting . . . . .                                | 25        |
| <b>7</b> | <b>Testing and Results</b>                         | <b>26</b> |
| 7.1      | Smoke Testing . . . . .                            | 26        |
| 7.2      | Evaluation Metrics . . . . .                       | 27        |
| 7.3      | Model Evaluation . . . . .                         | 27        |
| 7.4      | Sample Input Results . . . . .                     | 28        |

|          |                                    |           |
|----------|------------------------------------|-----------|
| 7.5      | Issues and Resolutions . . . . .   | 29        |
| 7.6      | Discussion . . . . .               | 29        |
| <b>8</b> | <b>Conclusion and Future Work</b>  | <b>30</b> |
| 8.1      | Conclusion . . . . .               | 30        |
| 8.2      | Future Enhancements . . . . .      | 30        |
| <b>A</b> | <b>Installation and User Guide</b> | <b>31</b> |
| A.1      | Environment Setup . . . . .        | 31        |
| A.2      | Running the Application . . . . .  | 31        |
| A.3      | CSV Batch Upload . . . . .         | 31        |
| A.4      | Testing Command . . . . .          | 32        |
|          | <b>References</b>                  | <b>32</b> |

## List of Figures

|     |                                                        |    |
|-----|--------------------------------------------------------|----|
| 4.1 | Agile Software Development Life Cycle Stages . . . . . | 10 |
| 4.2 | Prediction Workflow Flowchart . . . . .                | 15 |
| 5.1 | System Architecture . . . . .                          | 18 |
| 5.2 | Use Case Diagram . . . . .                             | 19 |
| 5.3 | Data Flow Diagram . . . . .                            | 20 |
| 5.4 | Login Page . . . . .                                   | 22 |
| 5.5 | Transaction Prediction Form . . . . .                  | 22 |
| 5.6 | Prediction Result View . . . . .                       | 23 |
| 5.7 | CSV Batch Upload . . . . .                             | 23 |
| 5.8 | Alert and Transaction Table . . . . .                  | 23 |

## List of Tables

|     |                                                      |    |
|-----|------------------------------------------------------|----|
| 3.1 | Functional Requirements . . . . .                    | 6  |
| 4.1 | Technology and Tools Used . . . . .                  | 11 |
| 4.2 | Roles and Responsibilities of Team Members . . . . . | 11 |
| 4.3 | Model Features . . . . .                             | 13 |
| 4.4 | Random Forest Training Configuration . . . . .       | 14 |
| 5.1 | Main Repository Components . . . . .                 | 19 |
| 5.2 | Application Routes . . . . .                         | 20 |
| 7.1 | Functional Test Cases . . . . .                      | 27 |
| 7.2 | Confusion Matrix . . . . .                           | 28 |
| 7.3 | Classification Report . . . . .                      | 28 |
| 7.4 | Issues Encountered and Resolutions . . . . .         | 29 |

# Chapter 1

## Introduction

### 1.1 Overview

Credit card fraud detection is a classification problem in which a system must identify whether a transaction is genuine or fraudulent. The difficulty of the task comes from class imbalance, changing fraud behavior, and the need for rapid decisions. The repository implements a practical fraud monitoring dashboard rather than only a notebook experiment. The main application is written in Flask and uses a trained Random Forest model to classify transactions.

The system is organized around the file `app.py`. It loads the model artifact `fraud_model.pkl` and the scaler artifact `scaler.pkl`, accepts transaction inputs, engineers the required features, predicts fraud probability, assigns a risk score, creates alerts, stores monitoring history, and exposes dashboard/reporting APIs. A shared module, `feature_engineering.py`, keeps training-time and inference-time feature definitions consistent.

### 1.2 Problem Statement

Manual inspection of card transactions is slow and unreliable when transaction volume is high. A fraud detection system must process transactions quickly, recognize risky behavior, reduce missed fraud cases, and still allow human review for high-risk or uncertain cases. The project addresses this problem by building an application that combines machine learning prediction with practical monitoring workflows.

The system must handle two types of input. First, an operator should be able to submit a single transaction through the dashboard and immediately see fraud probability, prediction label, risk score, behavior signals, and verification status. Second, an operator should be able to upload a CSV file containing multiple transactions and receive a processed summary with errors, alerts, and updated dashboard metrics.

### 1.3 Objectives

- Develop a Random Forest based credit card fraud detection model using engineered transaction and behavior features.

- Build a Flask-based monitoring dashboard that supports single and batch (CSV) fraud detection, along with risk scoring, alerts, verification workflow, and downloadable reports.

## 1.4 Major Features

- **Authentication:** Operators must sign in before accessing prediction and dashboard APIs.
- **Machine learning prediction:** A Random Forest classifier returns fraud probability and a binary label.
- **Behavior analysis:** Customer profiles track previous amounts, countries, devices, merchant categories, and recent transaction counts.
- **Risk scoring:** Model output and behavior signals are combined into a 0 to 100 risk score.
- **Real-time alerts:** Fraud predictions and high-risk transactions create alert records.
- **Verification workflow:** Operators can approve, block, review, or mark transactions as false positives.
- **CSV batch upload:** Multiple transactions can be processed from a structured CSV file.
- **Reports:** The system can export monitored transactions as `fraud_report.csv`.

## 1.5 Significance

The project is significant because credit card fraud can cause direct financial loss, customer inconvenience, and additional investigation workload for financial institutions. A system that can automatically score transactions helps analysts identify suspicious cases faster than manual review alone.

The implemented dashboard also demonstrates how a machine learning model can be converted into an operational workflow. Instead of stopping at model accuracy, the system provides fraud probability, risk level, behavior signals, alerts, verification status, CSV batch processing, and downloadable reports. These features make the project more practical for fraud monitoring because they support both automated screening and human decision-making.

For academic purposes, the project shows the complete path from data preparation and feature engineering to model training, deployment, testing, and documentation. It can

also serve as a base for future work using real banking data, stronger security controls, database-backed persistence, and model drift monitoring.

## 1.6 Scope

The scope of this project includes the training and serving of a Random Forest model, a web dashboard, authenticated APIs, persistence through a JSON store, and test coverage through the included smoke-test script. The implementation is suitable for academic demonstration and for explaining how a fraud monitoring workflow can be built around a machine learning model.

## 1.7 Limitations

- The repository uses `instance/data_store.json` for runtime persistence instead of a production database.
- Authentication uses simple username/password checking and should be strengthened before production use.
- The dataset in the repository is synthetic and may not capture all real banking fraud patterns.
- Model drift, audit logging, encryption, rate limiting, and role-based access control are outside the current implementation.
- The model threshold is configurable through `FRAUD_THRESHOLD`; when it is not set, the application loads the recommended threshold from `model_metadata.json`.

## 1.8 Organization of the Document

Chapter 2 presents the literature and conceptual background. Chapter 3 describes the requirements and feasibility. Chapter 4 explains the methodology, feature engineering, model training, and prediction workflow. Chapter 5 presents the system design. Chapter 6 describes implementation details from the repository. Chapter 7 summarizes testing and results. Chapter 8 concludes the project and lists future enhancements. The appendix provides installation and user-guide instructions.

## **Chapter 2**

### **Literature Review**

#### **2.1 Theoretical Review**

Credit card fraud detection is commonly modeled as a supervised binary classification task. Each transaction is represented by features such as amount, time, merchant behavior, channel, and geographical indicators. The target class indicates whether the transaction is fraudulent. A key challenge is class imbalance: fraud transactions usually represent only a small fraction of the full dataset, so a model can obtain high accuracy while still missing important fraud cases.

Fraud detection systems usually combine transaction-level features and behavior-level features. Transaction-level features describe the current event, such as amount, hour, country, and channel. Behavior-level features compare the current event against the customer profile, such as average previous amount, recent transaction count, new country, and new merchant category. This combination is important because fraud is often visible as a deviation from previous customer behavior.

#### **2.2 Empirical Review**

Previous research on credit card fraud detection has applied many machine learning methods, including Logistic Regression, Decision Trees, Random Forests, Support Vector Machines, Gradient Boosting models, Autoencoders, and probabilistic ensemble models [4–6]. These studies commonly report that class imbalance is one of the main difficulties in fraud detection because normal transactions heavily outnumber fraudulent transactions [1, 2].

Studies also show that accuracy alone is not enough for evaluating fraud detection. A model can achieve high accuracy by predicting most transactions as normal, while still missing a large number of fraud cases. Therefore, precision, recall, F1-score, F2-score, PR-AUC, ROC-AUC, and confusion matrix values are more meaningful for understanding fraud-class performance.



## 2.3 Machine Learning Techniques for Fraud Detection

Machine learning methods are useful because they can learn patterns from historical transaction data and generalize to new transactions. Logistic Regression is simple and interpretable, but may underfit nonlinear fraud patterns. Decision Trees are easy to explain, but can overfit. Support Vector Machines can work well on structured data, but may be harder to scale and tune. Neural networks can learn complex relationships, but require more data and careful training.

Random Forest is a suitable choice for this project because it is robust, handles nonlinear relationships, works well with tabular engineered features, and gives strong performance without requiring a very large deep-learning dataset [1, 2]. Random Forest is an ensemble learning method that builds many decision trees and combines their predictions. Each tree is trained on a bootstrapped sample of the data and considers a random subset of features at each split. This reduces overfitting compared with a single decision tree and improves generalization. In this project, `train_model.py` trains a `RandomForestClassifier` with 200 trees, class-weight balancing, maximum depth 12, and a fixed random seed for reproducibility.

## 2.4 Research Gaps

Many academic fraud detection projects focus mainly on training a model and reporting metrics. This is useful, but it does not fully represent the operational workflow needed by fraud analysts. In practice, analysts need a dashboard, authentication, transaction validation, risk scoring, alerts, verification actions, batch upload, and reports.

This project addresses that gap by connecting the machine learning model to a Flask-based monitoring dashboard. The repository extends model prediction with behavior profiling, risk calculation, alert generation, analyst verification, CSV batch detection, dashboard metrics, and report export. This makes the project more complete as a software system, not only as a model experiment.

## 2.5 Literature Summary

The reviewed work supports three design decisions in this project. First, fraud detection should be evaluated with fraud-sensitive metrics, not accuracy alone. Second, Random Forest is a practical baseline for tabular fraud features. Third, real-time and online payment fraud systems require an operational layer around the model, including monitoring, alerts, and review workflows [3, 7]. This project applies those ideas through a working dashboard, behavior profiling, batch upload, and report generation.

## Chapter 3

### System Analysis

#### 3.1 Existing System

Traditional fraud review relies on rule-based checks or manual investigation. These approaches can be useful, but they are limited when fraud behavior changes or transaction volume increases. Static rules may create many false positives or fail to identify new fraud patterns. Manual review also requires significant time and effort.

#### 3.2 Proposed System

The proposed system is a web-based credit card fraud detector named Fraud Shield. It uses a Random Forest model for prediction and adds a monitoring layer around the model. Each transaction passes through validation, behavior analysis, feature engineering, scaling, model prediction, risk scoring, alert generation, profile update, and persistence. Operators interact with the system through a browser dashboard and API endpoints.

#### 3.3 Functional Requirements

Table 3.1: Functional Requirements

| Requirement       | Description                                                                                                                               |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Login and logout  | The system must authenticate operators before protected routes are accessed.                                                              |
| Single prediction | The system must accept transaction details and return fraud probability, label, risk score, and behavior signals.                         |
| CSV upload        | The system must process CSV files containing multiple transactions.                                                                       |
| Dashboard metrics | The system must show total transactions, fraud predictions, high-risk transactions, open alerts, verified transactions, and average risk. |
| Alert handling    | The system must generate and acknowledge alerts for suspicious transactions.                                                              |

| Requirement   | Description                                                                                         |
|---------------|-----------------------------------------------------------------------------------------------------|
| Verification  | The system must allow transactions to be approved, blocked, reviewed, or marked as false positives. |
| Report export | The system must provide a downloadable CSV report of monitored transactions.                        |

## 3.4 Non-Functional Requirements

- **Usability:** The dashboard should be easy for an operator to use without command-line interaction.
- **Reliability:** Model and scaler artifacts must load successfully before prediction.
- **Maintainability:** Training and inference must share the same feature order and feature names.
- **Performance:** Predictions should be lightweight enough for real-time dashboard use.
- **Security:** Protected endpoints must reject unauthenticated requests.
- **Auditability:** Transaction records should include operator, verification status, behavior signals, and timestamps.

## 3.5 Feasibility Study

### 3.5.1 Technical Feasibility

The project is technically feasible because it uses well-supported Python libraries: Flask for the web application, Pandas and NumPy for data processing, Scikit-learn for machine learning, and Joblib for model persistence. The repository already contains trained artifacts and a smoke-test script, so the system can be run and tested locally.

### 3.5.2 Economic Feasibility

The tools used in this project are open source and do not require licensing fees. The system can run on a standard development machine, making it economically feasible for academic demonstration.

### 3.5.3 Operational Feasibility

The dashboard design supports operators by presenting prediction results, alerts, and verification actions in one place. CSV batch upload also makes the system practical for testing

multiple transactions without writing API calls manually.

### **3.5.4 Schedule Feasibility**

The project is schedule feasible because the work is divided into manageable phases: dataset preparation, feature engineering, model training, Flask application development, dashboard integration, testing, and report writing. The use of existing open-source libraries also reduces implementation time, allowing the main objectives to be completed within the Project-II semester timeline.

## **3.6 Ethical and Security Considerations**

Fraud detection systems influence financial decisions, so their output should support human review rather than act as the only decision authority. In this project, high-risk transactions are placed in a verification workflow where an operator can approve, block, review, or mark the transaction as a false positive. This reduces the risk of treating every model prediction as final.

The current implementation is intended for academic demonstration and does not store real card numbers or sensitive customer records. A production deployment would require stronger controls, including hashed passwords, role-based access control, encrypted transport, audit logging, database access controls, and privacy review before using real banking data.

# Chapter 4

## Methodology

### 4.1 Software Development Life Cycle

This project follows the Agile development approach, where the system development is divided into smaller iterative phases. The Agile methodology enables early feedback, continuous improvement, and flexibility to adapt to changing requirements. The development phases include:

- **Planning and Requirements Gathering:** Understanding project scope, objectives, and defining user stories for fraud monitoring features such as authentication, prediction, and verification.
- **Design:** Creating system architecture, data flow diagrams, and designing the dashboard interface optimized for transaction monitoring and fraud analysis.
- **Development:** Implementing core features in iterative sprints, including model training, feature engineering, backend APIs, dashboard UI, and CSV batch processing.
- **Testing:** Validating functionality and performance through smoke tests and verifying model predictions using precision, recall, and PR-AUC metrics.
- **Deployment:** Releasing the system as a runnable Flask application for local demonstration and fraud monitoring evaluation.
- **Review:** Gathering feedback from testing results and refining the system, with a focus on improving fraud detection accuracy and operational workflow.

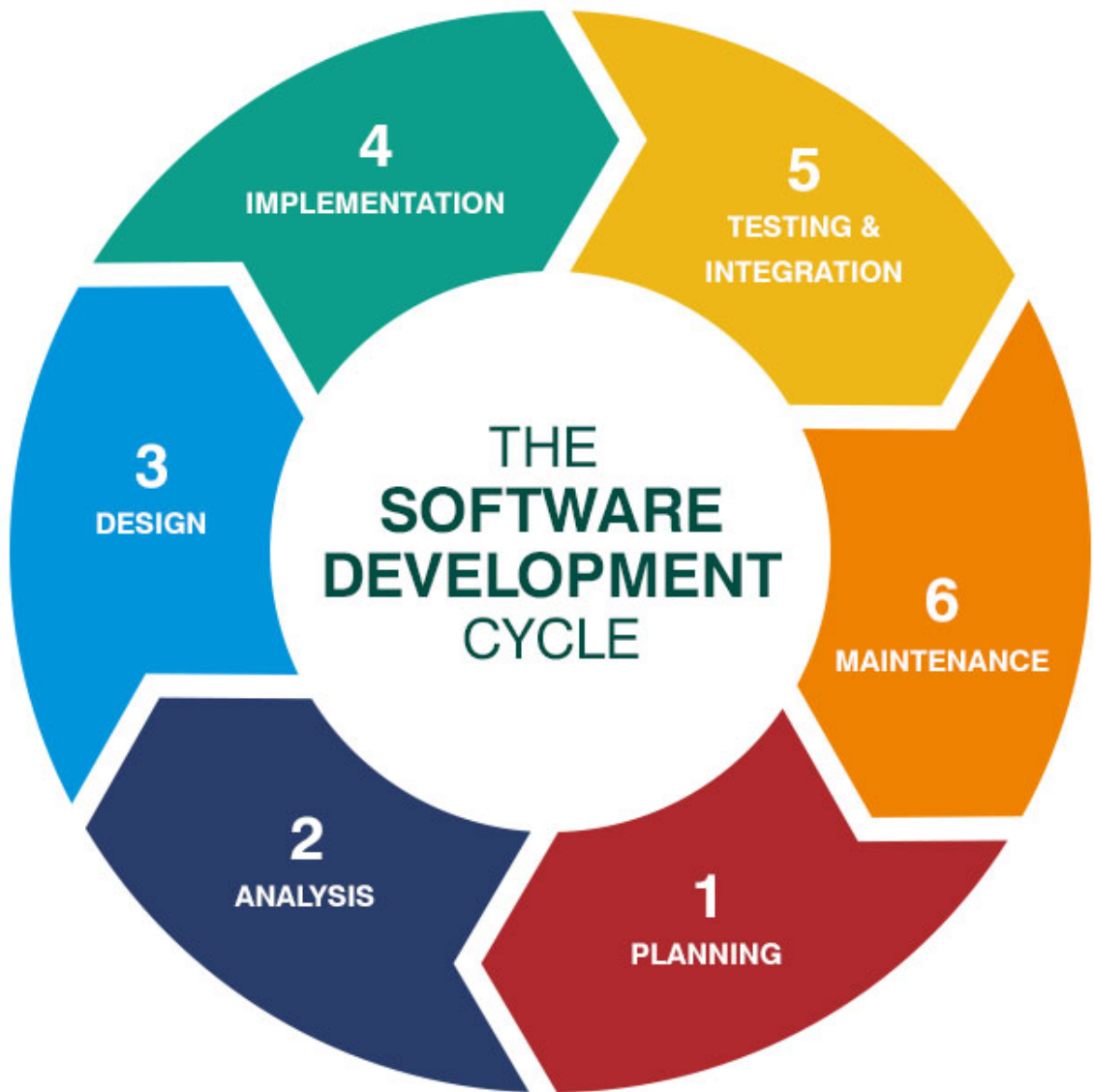


Figure 4.1: Agile Software Development Life Cycle Stages

## 4.2 Technology and Tools Used

Table 4.1: Technology and Tools Used

| Technology / Tool         | Purpose                                                                                          |
|---------------------------|--------------------------------------------------------------------------------------------------|
| Python                    | Main programming language for backend logic, data processing, and machine learning.              |
| Flask                     | Web application framework used for dashboard routes, APIs, authentication, and request handling. |
| Scikit-learn              | Machine learning library used for Random Forest training, scaling, metrics, and validation.      |
| Pandas and NumPy          | Data loading, preprocessing, feature computation, and numerical operations.                      |
| Joblib                    | Saving and loading trained model and scaler artifacts.                                           |
| HTML, CSS, and JavaScript | Frontend dashboard interface and user interaction.                                               |
| JSON                      | Lightweight runtime persistence for transactions, alerts, and behavior profiles.                 |
| LaTeX                     | Project report preparation and academic documentation.                                           |
| Visual Studio Code        | Development environment for editing, testing, and organizing source files.                       |

## 4.3 Assignment of Roles and Responsibilities

Table 4.2: Roles and Responsibilities of Team Members

| Team Member        | Primary Responsibilities                                                                                                                                                                           |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Santosh Kumar Shah | <ul style="list-style-type: none"><li>• Model training support.</li><li>• Feature engineering review.</li><li>• Prediction workflow testing.</li><li>• Documentation contribution.</li></ul>       |
| Surya Chand Yadav  | <ul style="list-style-type: none"><li>• Flask dashboard support.</li><li>• CSV batch workflow.</li><li>• API testing and smoke-test verification.</li><li>• Result analysis.</li></ul>             |
| Salim Shrestha     | <ul style="list-style-type: none"><li>• Report preparation.</li><li>• System design diagrams.</li><li>• Frontend review and literature review.</li><li>• Final documentation formatting.</li></ul> |

## 4.4 Dataset

The repository contains the dataset file `creditcard_raw.csv`. It has 59,978 transaction records. The target column is `is_fraud`. The dataset contains 922 fraud transactions, which is approximately 1.54 percent of the data. This imbalance is realistic for fraud detection and is handled through model training choices such as class-weight balancing and validation-based threshold selection.

## 4.5 Data Preparation

The training script validates that required transaction columns are present before model training. Transaction date and time are converted into a combined timestamp, records are ordered chronologically, and customer-history features are calculated using only previous transactions. This prevents the model from using future behavior when computing features for earlier transactions.

Categorical fields such as merchant category, channel, and country are converted into fixed integer encodings. Unknown values are mapped to an `other` category. Amount values are converted to floating-point numbers and transformed with `log1p` to reduce the effect of very large transaction amounts.

## 4.6 Input Fields

The application requires the following transaction fields for prediction:

- `transaction_date`
- `transaction_time`
- `amount`
- `merchant_category`
- `country`
- `channel`

Optional metadata fields include `customer_id`, `card_last4`, `merchant`, and `device_id`. These fields support behavior analysis, dashboard display, and report generation.



## 4.7 Feature Engineering

The file `feature_engineering.py` defines the model feature list and the function `engineer_single_transaction()`. It converts raw transaction data and behavior signals into the exact 15 features expected by the trained model.

Table 4.3: Model Features

| Feature               | Meaning                                                             |
|-----------------------|---------------------------------------------------------------------|
| hour                  | Hour extracted from transaction time.                               |
| day_of_week           | Day index extracted from transaction date.                          |
| is_weekend            | Indicates Saturday or Sunday.                                       |
| is_night              | Indicates transaction between 00:00 and 05:59.                      |
| amount                | Transaction amount.                                                 |
| amount_log            | Natural log transform of amount using $\log_{1p}$ .                 |
| merchant_category_enc | Encoded merchant category.                                          |
| channel_enc           | Encoded transaction channel.                                        |
| country_enc           | Encoded country.                                                    |
| txn_count_last_1h     | Previous customer transactions in the last hour.                    |
| txn_count_last_24h    | Previous customer transactions in the last 24 hours.                |
| avg_amount_prev       | Customer average amount before the current transaction.             |
| amount_ratio          | Current amount divided by previous average amount.                  |
| is_new_country        | Indicates a country not previously seen for the customer.           |
| is_new_category       | Indicates a merchant category not previously seen for the customer. |

## 4.8 Model Training

Training is implemented in `train_model.py`. The script loads `creditcard_raw.csv`, validates required columns, computes the engineered features, scales them with `StandardScaler`, splits the data using a chronological 70/15/15 train-validation-test split when all splits contain both classes, tunes a probability threshold on validation data, and trains a `RandomForestClassifier`.

Table 4.4: Random Forest Training Configuration

| Parameter             | Value                                                    |
|-----------------------|----------------------------------------------------------|
| Model                 | RandomForestClassifier                                   |
| Number of trees       | 200                                                      |
| Class weight          | balanced                                                 |
| Maximum depth         | 12                                                       |
| Random state          | 42                                                       |
| Parallel jobs         | -1                                                       |
| Split strategy        | Chronological 70/15/15 train-validation-test split       |
| Threshold selection   | F2 score on validation data with minimum precision guard |
| Recommended threshold | 0.371067                                                 |
| Cross-validation      | 5-fold TimeSeriesSplit on training data                  |

## 4.9 Prediction Pipeline

The application prediction pipeline is implemented by `monitor_transaction()` in `app.py`. The workflow is:

1. Validate and sanitize the incoming transaction.
2. Load the customer behavior profile before updating it.
3. Analyze behavior signals such as new device, new country, high amount, and transaction velocity.
4. Engineer the 15 model features.
5. Scale features using `scaler.pkl`.
6. Predict fraud probability using `fraud_model.pkl`.
7. Convert probability into a binary prediction using the configured threshold.
8. Calculate risk score and risk level.
9. Create an alert when the transaction is model-flagged or high risk.
10. Update the customer behavior profile.
11. Save transaction, alert, and profile state to `instance/data_store.json`.

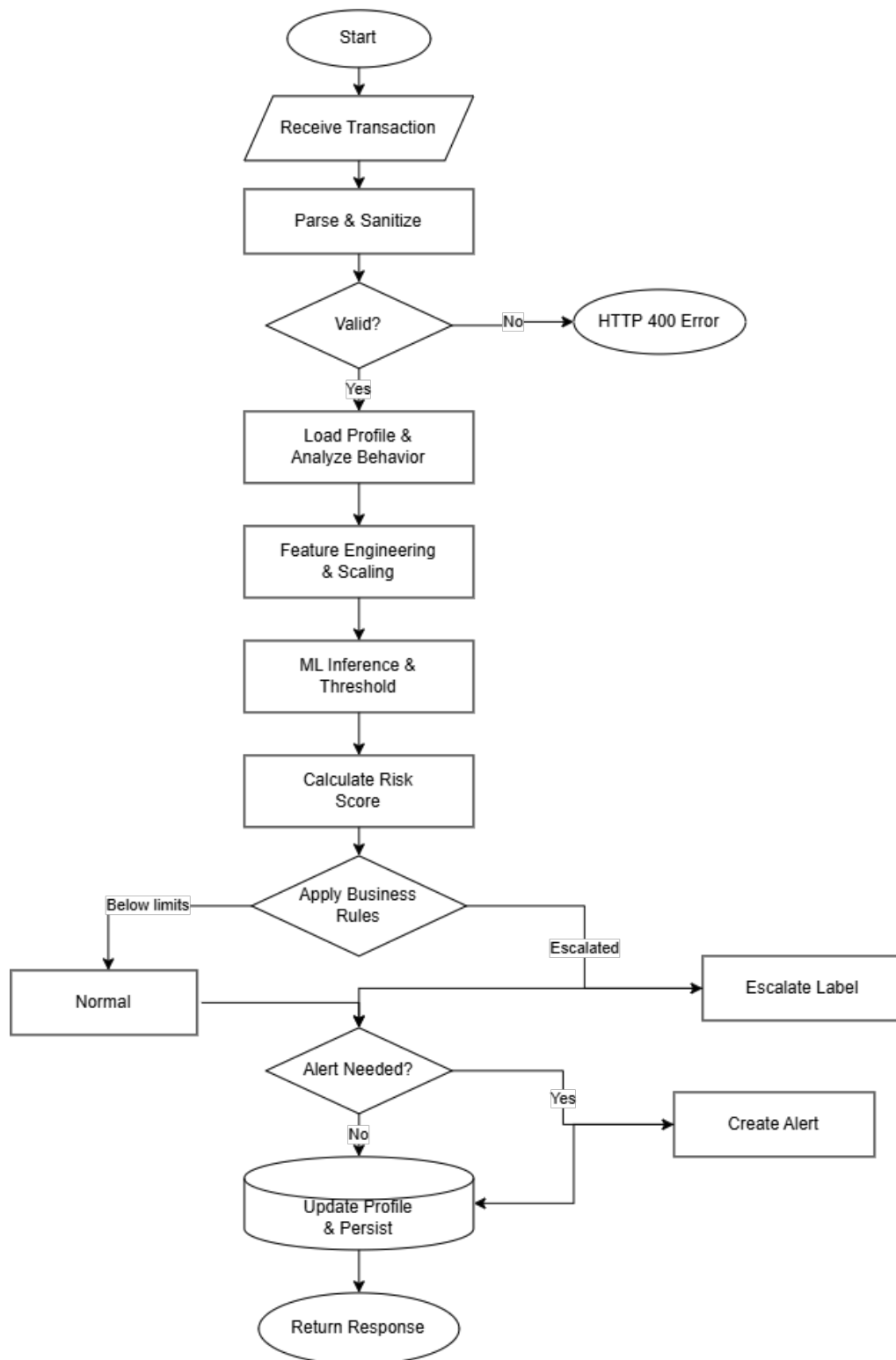


Figure 4.2: Prediction Workflow Flowchart

## 4.10 Risk Score Calculation

The risk score combines model probability, model decision, and behavior risk points. The implemented calculation is:

$$\text{risk score} = \min(100, \text{round}(72p + f + b))$$

where  $p$  is the model fraud probability,  $f$  is 10 when the model predicts fraud and 0 otherwise, and  $b$  is the behavior component capped at 28 points.

Risk levels are assigned as follows:

- Low: score below 35
- Medium: score from 35 to 64
- High: score from 65 to 84
- Critical: score 85 or above

## **Chapter 5**

### **System Design**

#### **5.1 Architecture**

The system follows a client-server architecture. The browser dashboard communicates with the Flask backend. The backend validates transactions, runs preprocessing, uses the machine learning artifacts, updates in-memory and JSON-based state, and returns JSON responses to the frontend.

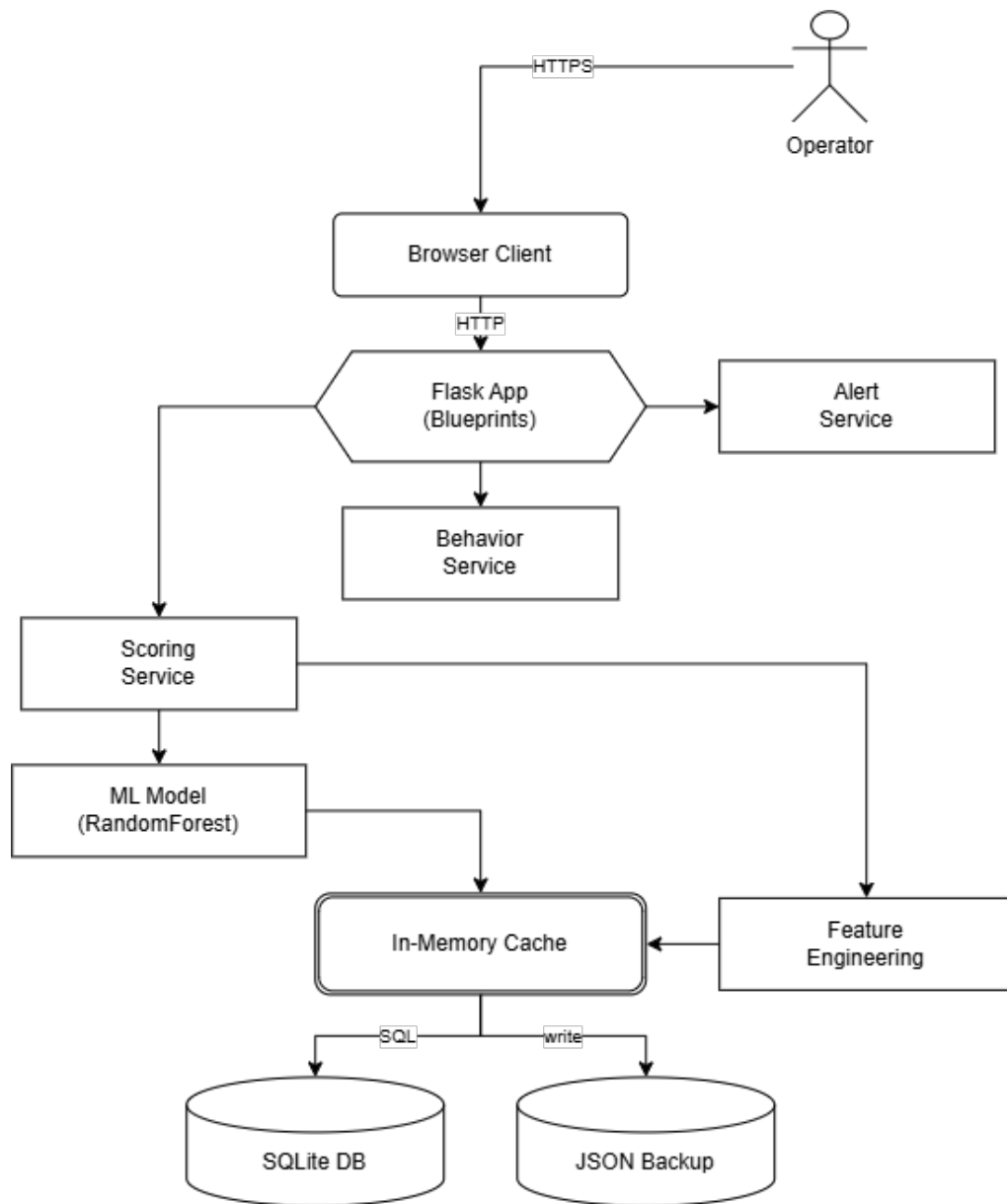


Figure 5.1: System Architecture

## 5.2 Use Case Design

The main actor of the system is the operator or administrator who reviews transactions and manages fraud alerts. The use case diagram in Figure 5.2 summarizes the primary dashboard functions available to this actor.

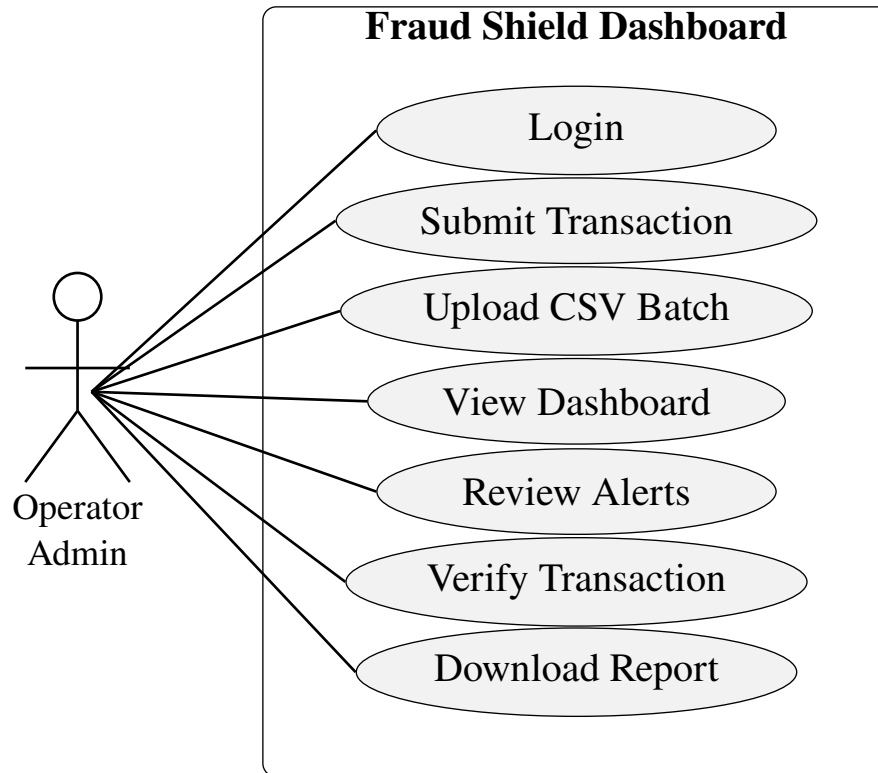


Figure 5.2: Use Case Diagram

Table 5.1: Main Repository Components

| Component                | Purpose                                                                                      |
|--------------------------|----------------------------------------------------------------------------------------------|
| app.py                   | Flask server, routes, authentication, prediction pipeline, alerts, reports, and persistence. |
| feature_engineering.py   | Shared feature definitions and single-transaction feature engineering.                       |
| train_model.py           | Retraining script for model and scaler artifacts.                                            |
| fraud_model.pkl          | Trained Random Forest classifier.                                                            |
| scaler.pkl               | Fitted StandardScaler for the 15 model features.                                             |
| templates/index.html     | Dashboard UI for login, prediction, CSV upload, alerts, and verification.                    |
| smoke_test.py            | End-to-end checks for the Flask application.                                                 |
| sample_upload.csv        | Example CSV file for batch detection.                                                        |
| sample_transactions.json | Example normal and fraud transaction payloads.                                               |
| instance/data_store.json | Runtime persistence for transactions, alerts, and behavior profiles.                         |

## 5.3 Data Flow Design

The data flow diagram shows how transaction data moves through the dashboard, backend, machine learning pipeline, alert workflow, and runtime storage. It complements the prediction workflow flowchart by showing the main data stores and outputs used by the complete monitoring system.

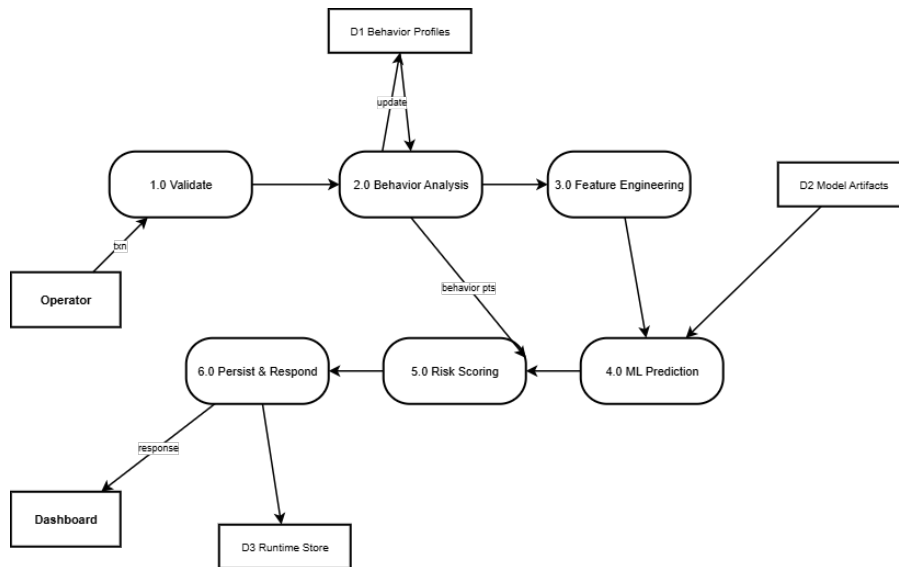


Figure 5.3: Data Flow Diagram

## 5.4 API Design

Table 5.2: Application Routes

| Method | Endpoint          | Purpose                                                              |
|--------|-------------------|----------------------------------------------------------------------|
| GET    | /                 | Render login page or dashboard.                                      |
| POST   | /login            | Authenticate operator.                                               |
| POST   | /logout           | Clear session.                                                       |
| GET    | /health           | Return application status, threshold, model files, and feature list. |
| POST   | /predict          | Process one transaction and return fraud/risk result.                |
| GET    | /api/dashboard    | Return dashboard metrics, recent transactions, alerts, and profiles. |
| GET    | /api/transactions | Return monitored transactions.                                       |
| GET    | /api/alerts       | Return active alerts.                                                |



| Method | Endpoint                                  | Purpose                                 |
|--------|-------------------------------------------|-----------------------------------------|
| POST   | /api/alerts/<alert_id>/acknowledge        | Mark an alert as acknowledged.          |
| POST   | /api/transactions/<transaction_id>/verify | Update transaction verification status. |
| GET    | /api/sample-csv                           | Download sample batch upload CSV.       |
| POST   | /api/upload-csv                           | Process uploaded CSV transactions.      |
| GET    | /api/report.csv                           | Download monitored transaction report.  |

## 5.5 Persistence Design

The application stores runtime state in memory and writes it to `instance/data_store.json`. If an older root-level `data_store.json` file exists, the application can read it as a legacy state file, but new writes use the `instance` directory. The state includes monitored transactions, alerts, and behavior profiles. Profiles contain transaction counts, amount statistics, recent timestamps, known devices, known countries, and known merchant categories. This design is simple and adequate for a demonstration project, but a production system should use a database.

## 5.6 User Interface Design

The dashboard in `templates/index.html` contains the following major areas:

- Login panel for unauthenticated users.
- Transaction monitoring form for single prediction.
- Prediction result cards for label, fraud probability, risk score, and verification status.
- CSV upload panel for batch fraud detection.
- Behavior signals and preprocessing summary.
- Alert queue and dashboard metrics.
- Recent transaction table with verification actions.

## 5.7 User Interface Screenshots

The following screenshots show the main operator-facing screens of the implemented dashboard.

# Fraud Shield

Secure monitoring console

**Username**

admin

**Password**

.....

Sign In

Figure 5.4: Login Page

## Fraud Shield Dashboard

Operator: admin | Decision threshold: 0.5

### Transaction Monitoring

|                  |             |               |           |
|------------------|-------------|---------------|-----------|
| Customer ID      | Card Last 4 | Device ID     |           |
| CUST-001         | 4242        | web-laptop-01 |           |
| Transaction Date | Time        | Amount        | Merchant  |
| 2024-01-15       | 09:23:44    | 45.20         | SuperMart |
| Category         | Country     | Channel       |           |
| Grocery          | US          | Web           |           |

Run Monitoring Check

Load Normal

Load Fraud

Clear

Figure 5.5: Transaction Prediction Form

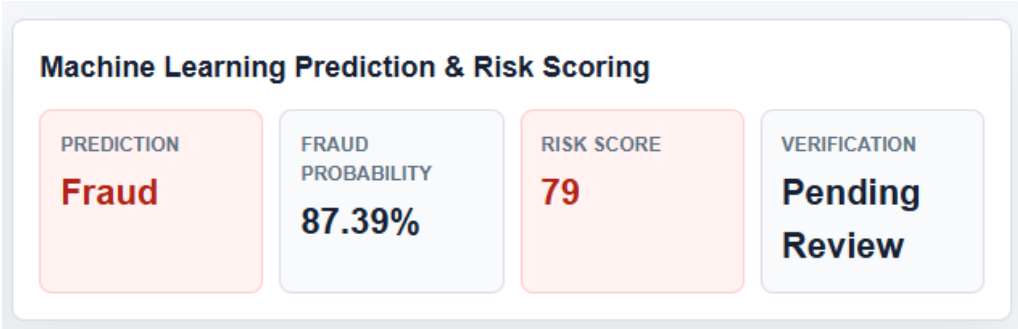


Figure 5.6: Prediction Result View

**Manual CSV Fraud Detection**

Upload a CSV with the required transaction columns (transaction\_date, transaction\_time, amount, merchant\_category, country, channel). Each row will be preprocessed, scored, risk-ranked, monitored, and added to alerts/reports.

CSV File

No file chosen

No CSV batch scanned yet.

Figure 5.7: CSV Batch Upload

**User Verification Queue**

| TRANSACTION                           | CUSTOMER                | AMOUNT | ML              | RISK    | STATUS         | ACTION                                 |                                      |                                               |
|---------------------------------------|-------------------------|--------|-----------------|---------|----------------|----------------------------------------|--------------------------------------|-----------------------------------------------|
| 22ae2ab8<br>2026-05-20T17:58:40+00:00 | CUST-001<br>SuperMart   | 45.20  | Normal<br>0.00% | 0 Low   | Auto Cleared   | <input type="button" value="Approve"/> | <input type="button" value="Block"/> | <input type="button" value="False Positive"/> |
| 3c214290<br>2026-05-20T17:58:37+00:00 | CUST-009<br>Unknown ATM | 500.00 | Fraud<br>87.39% | 79 High | Pending Review | <input type="button" value="Approve"/> | <input type="button" value="Block"/> | <input type="button" value="False Positive"/> |
| 47d1d073<br>2026-05-20T17:56:51+00:00 | CUST-009<br>Unknown ATM | 500.00 | Fraud<br>87.39% | 79 High | Pending Review | <input type="button" value="Approve"/> | <input type="button" value="Block"/> | <input type="button" value="False Positive"/> |
| 08c4390e<br>2026-05-20T16:57:39+00:00 | CUST-001<br>SuperMart   | 45.20  | Normal<br>0.00% | 0 Low   | Auto Cleared   | <input type="button" value="Approve"/> | <input type="button" value="Block"/> | <input type="button" value="False Positive"/> |
| d536b7b7<br>2026-05-20T16:57:37+00:00 | CUST-009<br>Unknown ATM | 500.00 | Fraud<br>88.39% | 80 High | Pending Review | <input type="button" value="Approve"/> | <input type="button" value="Block"/> | <input type="button" value="False Positive"/> |
| 69afb8f8<br>2026-05-20T16:15:19+00:00 | CUST-009<br>Unknown ATM | 500.00 | Fraud<br>89.89% | 81 High | Pending Review | <input type="button" value="Approve"/> | <input type="button" value="Block"/> | <input type="button" value="False Positive"/> |
| e76bff0b<br>2026-05-20T16:13:25+00:00 | CUST-009<br>Unknown ATM | 500.00 | Fraud<br>91.37% | 82 High | Pending Review | <input type="button" value="Approve"/> | <input type="button" value="Block"/> | <input type="button" value="False Positive"/> |

Figure 5.8: Alert and Transaction Table

## Chapter 6

### Implementation

#### 6.1 Development Environment

The project is implemented in Python. The required dependencies are listed in `requirements.txt`: Flask, Pandas, NumPy, Scikit-learn, and Joblib. The implementation is organized into separate files for the Flask application, feature engineering, model training, synthetic data generation, smoke testing, templates, and saved model artifacts.

#### 6.2 Authentication

The application uses session-based authentication. The username, password, and secret key can be configured through environment variables. Protected endpoints use the `require_auth` decorator, which returns HTTP 401 for unauthenticated requests.

#### 6.3 Transaction Validation

The function `parse_transaction()` validates required fields and ensures that `amount` is a non-negative finite number. It also fills optional metadata with safe defaults when values are missing. This prevents the model pipeline from receiving incomplete or malformed transaction objects.

#### 6.4 Feature Encoding

The feature engineering module contains fixed category lists for merchant categories, channels, and countries. Unknown values are mapped to the final `other` category. The order of these lists is important because the encoded integer values are part of the model input.

#### 6.5 Behavior Analysis

The function `analyze_behavior()` computes behavior points and signal descriptions. It checks whether the customer is new, whether the amount is much higher than previous

average, whether the device, country, or merchant category is new, whether recent transaction count is high, whether the amount is large, and whether the transaction occurs at night.

## 6.6 Prediction and Risk

The `score_ml()` function calls `predict_proba()` on the Random Forest model and uses the configured threshold to decide whether the transaction is fraud. The `calculate_risk()` function combines model probability and behavior points to produce the final operational risk score. This means an unusual but model-benign transaction can still become high risk if behavior indicators are strong.

## 6.7 Alerts and Verification

The system creates an alert when the transaction is predicted as fraud or when the risk score is at least 65. High-risk transactions are placed in `PendingReview`. Operators can update verification status through the dashboard as `Approved`, `Blocked`, `Reviewed`, or `False Positive`. When a transaction is verified, related alerts are acknowledged.

## 6.8 CSV Batch Detection

The CSV upload endpoint reads an uploaded file with Pandas, enforces a maximum of 500 rows, checks required columns, and processes each row through the same monitoring pipeline used by single prediction. Each successful row returns transaction ID, customer ID, amount, label, fraud probability, risk score, risk level, and verification status.

## 6.9 Reporting

The report endpoint creates a CSV file with transaction ID, timestamp, customer ID, merchant, amount, fraud probability, model prediction, risk score, risk level, verification status, and behavior signals. This makes the dashboard output easy to preserve or analyze in spreadsheet software.

## Chapter 7

### Testing and Results

#### 7.1 Smoke Testing

The repository includes `smoke_test.py`, which uses the Flask test client for end-to-end checks. The script verifies:

- `/health` returns status 200 and a valid feature count.
- Login succeeds with correct credentials and fails with wrong credentials.
- A normal sample transaction returns the expected Normal label.
- A fraud sample transaction returns the expected Fraud label.
- Fraud probability is within the range 0 to 1.
- Risk score is within the range 0 to 100.
- Behavior and preprocessing fields are present.
- Dashboard, sample CSV, CSV upload, report download, logout, and authentication checks work.

Table 7.1: Functional Test Cases

| Test Case         | Input / Action                        | Expected Result                                 | Actual Result                               | Status |
|-------------------|---------------------------------------|-------------------------------------------------|---------------------------------------------|--------|
| Feature encoding  | Encode US and lowercase ru.           | Correct country indexes are returned.           | Indexes matched expected values.            | Pass   |
| Health check      | Request /health.                      | HTTP 200 with status ok and feature count.      | Endpoint returned valid health data.        | Pass   |
| Valid login       | Submit correct username and password. | User is authenticated successfully.             | Login returned status ok.                   | Pass   |
| Invalid login     | Submit wrong password.                | Request is rejected with HTTP 401.              | Login failed with HTTP 401.                 | Pass   |
| Normal prediction | Submit normal sample transaction.     | Label is Normal with valid score fields.        | Normal label and valid scores returned.     | Pass   |
| Fraud prediction  | Submit fraud sample transaction.      | Label is Fraud with valid score fields.         | Fraud label and valid scores returned.      | Pass   |
| Validation error  | Submit invalid transaction date.      | Request is rejected with HTTP 400.              | Invalid date returned HTTP 400.             | Pass   |
| CSV upload        | Upload sample_upload.csv.             | Rows are processed with zero sample failures.   | Records returned and failed count was zero. | Pass   |
| Dashboard         | Request authenticated dashboard API.  | Metrics, transactions, and alerts are returned. | Dashboard payload was returned.             | Pass   |
| Report download   | Request /api/report.csv.              | Non-empty CSV response is downloaded.           | CSV response was generated.                 | Pass   |
| Logout protection | Logout and request dashboard again.   | Protected route rejects unauthenticated user.   | Dashboard returned HTTP 401.                | Pass   |

## 7.2 Evaluation Metrics

The evaluation uses metrics that are suitable for imbalanced fraud detection:

- **Precision:** the proportion of predicted fraud transactions that are actually fraud.
- **Recall:** the proportion of actual fraud transactions detected by the model.
- **F1-score:** the harmonic mean of precision and recall.
- **F2-score:** a recall-weighted score that gives more importance to detecting fraud cases.
- **PR-AUC:** area under the precision-recall curve, useful when fraud cases are rare.
- **ROC-AUC:** area under the receiver operating characteristic curve.

## 7.3 Model Evaluation

The model performance reported in `model_metadata.json` is based on the current chronological test split. The dataset contains 59,978 synthetic transactions with 922 fraud transactions. Therefore, the following results evaluate consistency with the synthetic data-generation process and should not be generalized to real banking operations without valida-

tion on representative, privacy-approved real-world data. The reported confusion matrix is shown below.

Table 7.2: Confusion Matrix

|                      | <b>Predicted Normal</b> | <b>Predicted Fraud</b> |
|----------------------|-------------------------|------------------------|
| <b>Actual Normal</b> | 8773                    | 86                     |
| <b>Actual Fraud</b>  | 44                      | 94                     |

Table 7.3: Classification Report

| <b>Class</b>     | <b>Precision</b> | <b>Recall</b> | <b>F1-score</b> | <b>Support</b> |
|------------------|------------------|---------------|-----------------|----------------|
| Normal           | 0.995            | 0.990         | 0.993           | 8859           |
| Fraud            | 0.522            | 0.681         | 0.591           | 138            |
| Accuracy         | 0.986            |               |                 | 8997           |
| Macro average    | 0.759            | 0.836         | 0.792           | 8997           |
| Weighted average | 0.988            | 0.986         | 0.986           | 8997           |

The repository metadata reports 5-fold time-series cross-validation on training data with a mean fraud recall of  $0.5467 \pm 0.1177$  and a mean PR-AUC of  $0.6374 \pm 0.0417$ . These results show that the model performs well for normal-class separation, but fraud-class recall remains an important improvement area because the dataset is highly imbalanced.

## 7.4 Sample Input Results

The repository includes two sample JSON payloads. The normal sample uses a grocery transaction of 45.20 in the United States. The fraud sample uses a cash transaction of 500.00 from Russia at night on an unknown device. The smoke test expects the first sample to be classified as Normal and the second as Fraud.



## 7.5 Issues and Resolutions

Table 7.4: Issues Encountered and Resolutions

| Issue                                          | Resolution                                                                                                                                                             |
|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Class imbalance in fraud data                  | <ul style="list-style-type: none"><li>• Used class-weight balancing in Random Forest training and selected a probability threshold using validation metrics.</li></ul> |
| Training and inference feature mismatch risk   | <ul style="list-style-type: none"><li>• Centralized feature names and single-transaction engineering in <code>feature_engineering.py</code>.</li></ul>                 |
| Unreliable first-transaction behavior features | <ul style="list-style-type: none"><li>• Used neutral behavior values for new customer profiles so the current transaction does not distort its own score.</li></ul>    |
| CSV input errors                               | <ul style="list-style-type: none"><li>• Added required-column validation, row limits, date/time normalization, and per-row error reporting.</li></ul>                  |
| False positive handling                        | <ul style="list-style-type: none"><li>• Added analyst verification statuses such as Approved, Blocked, Reviewed, and False Positive.</li></ul>                         |
| Runtime persistence limitation                 | <ul style="list-style-type: none"><li>• Used JSON persistence for demonstration and listed relational database migration as future work.</li></ul>                     |
| Operational visibility                         | <ul style="list-style-type: none"><li>• Added dashboard metrics, alerts, transaction history, and downloadable reports.</li></ul>                                      |

## 7.6 Discussion

The combination of model probability and behavior analysis is important. The Random Forest provides the statistical prediction, while behavior scoring provides operational context. This design supports practical fraud review by exposing both the model output and the reasons a transaction may be risky.

## Chapter 8

### Conclusion and Future Work

#### 8.1 Conclusion

The project successfully implements a credit card fraud detection system using a Random Forest model and a Flask dashboard. The repository demonstrates an end-to-end workflow: data preparation, feature engineering, model training, prediction serving, risk scoring, alerting, verification, CSV batch detection, and reporting. By using shared feature definitions, the project reduces the risk of mismatch between model training and application inference.

The system is suitable for academic demonstration because it is runnable, testable, and connected to an operator-facing interface. The model evaluation shows useful performance on the available synthetic dataset, and the dashboard provides the workflow needed to inspect, verify, and report suspicious transactions. Validation with representative real-world data is required before making claims about production fraud-detection performance.

#### 8.2 Future Enhancements

- Replace JSON-file persistence with PostgreSQL or MySQL for reliable storage.
- Add hashed passwords, role-based access control, and stronger session security.
- Add audit logs for all alert and verification actions.
- Add model versioning and store which model scored each transaction.
- Monitor model drift and retrain with real-world transaction data.
- Add explainability features such as feature importance or per-transaction explanations.
- Add API rate limiting and HTTPS deployment configuration.
- Expand testing with unit tests for feature engineering and risk scoring.

# Appendix A

## Installation and User Guide

### A.1 Environment Setup

The project requires Python and the packages listed in `requirements.txt`. From the project root directory, the environment can be prepared with:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

### A.2 Running the Application

The dashboard can be started with:

```
python app.py
```

After the Flask server starts, the dashboard is available at:

```
http://127.0.0.1:5000/
```

The default local login credentials are:

Username: admin

Password: admin123

For a real deployment, these credentials should be replaced using environment variables:

```
$env:APP_USERNAME = "analyst"
$env:APP_PASSWORD = "strong-password"
$env:SECRET_KEY = "replace-with-a-random-secret"
```

### A.3 CSV Batch Upload

The CSV upload workflow expects transaction records with fields such as customer ID, card number suffix, merchant, merchant category, country, device ID, channel, transaction date,

transaction time, and amount. A working example is provided in `sample_upload.csv`, and a downloadable sample is also exposed by the route `/api/sample-csv`.

## **A.4 Testing Command**

The smoke test can be run from the project root with:

```
python smoke_test.py
```

## References

- [1] Manoj Kumar Reddy Mallidi and Yeshwanth Zagabathuni. Analysis of credit card fraud detection using machine learning models on balanced and imbalanced datasets. *International Journal of Emerging Trends in Engineering Research*, 9(7):846–852, 2021.
- [2] Mayowa Timothy Adesina and Luke Howe. Credit card fraud detection. *International Journal of Science and Research Archive*, 12(2):2072–2080, 2024.
- [3] Faisal Tadvī, Sejal Shinde, Devendra Patil, and Sejal Dmello. Real time credit card fraud detection. *International Research Journal of Engineering and Technology*, 8(5), 2021.
- [4] Yihong He. Machine learning methods for credit card fraud detection. In *Highlights in Science, Engineering and Technology*, volume 23, Davis, CA, USA, 2022. Presented at IPIIS 2022.
- [5] Aditya Oza. Fraud detection using machine learning. Course project report, Stanford University, 2019.
- [6] Tzu-Hsuan Lin and Jehn-Ruey Jiang. Credit card fraud detection with autoencoder and probabilistic random forest. *Mathematics*, 9(21):2683, 2021.
- [7] Abdulwahab Ali Almazroi and Nasir Ayub. Online payment fraud detection model using machine learning techniques. *IEEE Access*, 2023.